DDD Tactical Design & Aggregate

1. Tactical Design là gì?

Strategic Design (chiến lược) = chia hệ thống lớn thành **Bounded Context** (vd: Job, Candidate, Application).

Tactical Design (chiến thuật) = **bên trong mỗi Bounded Context**, dùng pattern gì để code domain logic? Gồm 6 building block, **quan trọng nhất là Aggregate**.

Tactical Design gồm:

- Entity
- Value Object
- **Aggregate**
- Repository
- Domain Service
- Factory
- Domain Event

Trong đó: **Aggregate** chính là trung tâm điều phối business consistency.

2. Aggregate - Trái tim của Tactical Design

Định nghĩa đơn giản

Aggregate = **cụm objects liên quan** được xử lý như **1 đơn vị duy nhất** (1 transaction = 1 aggregate).

Ví dụ 1 (cơ bản): Đơn hàng (Order)

Hãy nghĩ về **1 đơn hàng** ở cửa hàng:

- Có **1 mã đơn** (Order #123)
- Chứa **nhiều dòng sản phẩm** (OrderLine: 2 áo + 1 quần)
- Có **địa chỉ giao hàng** (ShippingAddress)
- Có **trạng thái** (status = PENDING/SHIPPED/DELIVERED)

Order = Aggregate Root. OrderLine, ShippingAddress, status = các thành phần bên trong.

Ví dụ 2: Tài khoản ngân hàng (BankAccount)

Bạn có 1 tài khoản ngân hàng:

- **Số tài khoản** (accountNumber - identity)
- **Số dư** (balance)
- **Danh sách giao dịch** (List<Transaction>: nạp, rút, chuyển khoản)
- **Trạng thái** (status = ACTIVE/FROZEN/CLOSED)

BankAccount = Aggregate Root. Transaction là thành phần bên trong.

Quy tắc nghiệp vụ (invariants):

- Số dư không được âm
- Tài khoản FROZEN không cho rút tiền
- Mỗi giao dịch rút tiền phải tạo Transaction tương ứng

```
public class BankAccount {
    private String accountNumber;
    private BigDecimal balance;
    private List<Transaction> transactions = new ArrayList<>();
    private Status status;

    public void withdraw(BigDecimal amount) {
        if (status == Status.FROZEN) {
            throw new IllegalStateException("Tài khoản bị đóng băng");
        }
        if (balance.compareTo(amount) < 0) {
            throw new IllegalStateException("Số dư không đủ");
        }
        this.balance = balance.subtract(amount);
        transactions.add(Transaction.withdraw(amount));
    }
}
```

: **Bên ngoài không cách nào "lấy số dư âm"** vì mọi thao tác đều qua method có check.

Ví dụ 3: Bài viết Blog (BlogPost)

Blog có 1 bài viết:

- **ID bài viết** (postId)
- **Tiêu đề + nội dung** (title, content)
- **Danh sách comment** (List<Comment>)
- **Danh sách tag** (Set<Tag>)
- **Trạng thái** (status = DRAFT/PUBLISHED/ARCHIVED)

BlogPost = Aggregate Root. Comment + Tag là thành phần bên trong.

Quy tắc nghiệp vụ:

- DRAFT không cho comment
- PUBLISHED không sửa được content (chỉ thêm comment)
- ARCHIVED không cho thêm comment mới

```
public class BlogPost {
    private UUID postId;
    private String title;
    private String content;
    private List<Comment> comments = new ArrayList<>();
    private Status status;

    public void publish() {
        if (status != Status.DRAFT) {
            throw new IllegalStateException("Chỉ DRAFT mới publish
được");
        }
        this.status = Status.PUBLISHED;
    }

    public void addComment(String author, String text) {
        if (status != Status.PUBLISHED) {
            throw new IllegalStateException("Chỉ comment được khi
PUBLISHED");
        }
        comments.add(new Comment(author, text));
    }
}
```

: Quy tắc nghiệp vụ **luôn được bảo vệ** trong BlogPost. Service không cần check lại.

Quy tắc vàng

Quy tắc	Giải thích
1 Root duy nhất	Bên ngoài chỉ chạm vào Root (Order/BankAccount/BlogPost), KHÔNG chạm trực tiếp OrderLine/Transaction/Comment
1 transaction = 1 aggregate	Trong 1 `@Transactional`, chỉ update 1 Aggregate. Nếu dính dáng Aggregate khác -> dùng Domain Event.
Save cả cụm (Atomic)	Thay đổi entity con phải thông qua Root & gọi `RootRepo.save()`. Tuyệt đối KHÔNG bypass Root bằng Repository của entity con.
Invariants protection	Root đảm bảo các quy tắc nghiệp vụ luôn đúng (vd: tổng tiền = sum OrderLine)
Đặt method nghiệp vụ trên Root	KHÔNG có public setter - sửa state qua method tên có ý nghĩa

(1) 1 transaction = 1 aggregate

Trong kiến trúc phần mềm, Aggregate được định nghĩa là ranh giới của tính nhất quán dữ liệu (Consistency Boundary).

Quy tắc:

Trong một **Database Transaction** (`@Transactional` của Spring), bạn chỉ được phép cập nhật trạng thái của DUY NHẤT một Aggregate.

- **Sai lầm thường gặp**: Viết một hàm `@Transactional` để vừa lưu Order (Aggregate 1) vừa update số dư CustomerWallet (Aggregate 2) cùng lúc.
- **Tại sao phải làm vậy?**
  - **Hiệu suất & Scale**: Việc lock nhiều bảng/document cùng lúc để đảm bảo tính ACID sẽ gây ra thắt cổ chai (bottleneck) và deadlock khi hệ thống scale.
  - **Tiền đề cho Microservices**: Nếu hôm nay bạn update 2 Aggregates trong cùng 1 transaction, ngày mai khi tách 2 Aggregates đó ra 2 Microservices khác nhau (mỗi service 1 database riêng), code của bạn sẽ vỡ nát vì không thể chạy Distributed Transaction (2PC) một cách hiệu quả.
  - **Giải pháp**: Nếu nghiệp vụ yêu cầu đổi cả 2, hãy update Aggregate 1 -> lưu DB -> phát ra một sự kiện (Domain Event/Kafka) -> Aggregate 2 lắng nghe sự kiện đó và tự update trong một transaction khác (cơ chế Eventual Consistency).

(2) Lưu ý: Save cả cụm (Atomic Update)

Một Aggregate thường bao gồm 1 Root (ví dụ: `Application`) và nhiều Entity con bên trong (ví dụ: `Interview`). Bạn phải xem toàn bộ Aggregate đó là **1 khối nguyên tử (Atomic)**.

- **Cách thao tác ĐÚNG**:
  1. Load toàn bộ Aggregate từ DB: `app = ApplicationRepo.findById(id)`
  2. Sửa dữ liệu thông qua hàm của Root: `app.addInterview(newInterview)` *(Root tự động validate nghiệp vụ)*.
  3. Lưu toàn bộ khối đó lại thông qua Root: `ApplicationRepo.save(app)` *(JPA Cascade tự động lưu các record con)*.
- **Cách thao tác SAI (sửa nửa vời)**:
  - Tạo một `InterviewRepository` riêng biệt và gọi `InterviewRepository.save(newInterview)`.
  - **Hậu quả**: Root `Application` bị bypass, các quy tắc nghiệp vụ không được kiểm tra, dẫn đến dữ liệu mất nhất quán.

---

3. So sánh: Anemic Model (sai) vs Rich Model (đúng)

❌ Anemic Model - Chỉ là data bag (sai theo DDD)

```
public class Order {
    private OrderStatus status;
    public OrderStatus getStatus() { return status; }
    public void setStatus(OrderStatus s) { this.status = s; }  // ← Vấn
đề
}

// Logic nằm ở Service (procedural)
@Service
public class OrderService {
    public void cancel(Order order) {
        order.setStatus(OrderStatus.CANCELLED);  // ← Ai cũng set được,
kể cả đơn đã SHIPPED!
    }
}
```

Vấn đề:

- Ai cũng có thể `order.setStatus(SHIPPED)` rồi `order.setStatus(PENDING)` : vi phạm logic.

- Logic nghiệp vụ rời rạc trong Service : khó tìm "quy tắc cancel order ở đâu?".

✅ Rich Model - Object tự bảo vệ chính nó

```
public class Order {
    private OrderStatus status;

    public void cancel() {                                    // ←
Method có tên ý nghĩa
        if (status == OrderStatus.SHIPPED) {
            throw new IllegalStateException("Đơn đã giao, không thể
huỷ");
        }
        this.status = OrderStatus.CANCELLED;
    }

    public OrderStatus getStatus() { return status; }
    // KHÔNG có setStatus!
}

// Service chỉ điều phối, không chứa logic
@Service
public class CancelOrderUseCase {
    public void execute(OrderId id) {
        Order order = repo.findById(id);
        order.cancel();           // ← Quy tắc nằm trong Order
        repo.save(order);
    }
}
```

Lợi ích:

- KHÔNG ai có thể bypass quy tắc - Order tự kiểm tra.
- Đọc code biết ngay quy tắc nghiệp vụ ở đâu.
- Dễ test (mock Service không cần, chỉ test Order class).

---

4. Áp dụng vào TalentHub

Ví dụ 1: Aggregate `Job` (job-service)

Quy tắc nghiệp vụ (từ ATS SRS):

- **BRULE-14:** `minSalary` <= `maxSalary`, cả hai > 0
- **BRULE-11:** State machine: `DRAFT : PENDING_APPROVAL : APPROVED : PUBLISHED : CLOSED`

- **BRULE-13:** Khi PUBLISHED, không sửa được title/salary

```
public class Job {

    public enum Status { DRAFT, PENDING_APPROVAL, APPROVED, PUBLISHED,
CLOSED }

    private UUID id;
    private String title;
    private BigDecimal minSalary;
    private BigDecimal maxSalary;
    private Status status;

    private Job() { }   // private - chặn instantiate trực tiếp

    public static Job draft(String title, BigDecimal minSalary,
BigDecimal maxSalary) {
        if (minSalary.signum() <= 0 || maxSalary.signum() <= 0) {
            throw new IllegalArgumentException("Salary phải > 0");
        }
        if (minSalary.compareTo(maxSalary) > 0) {
            throw new IllegalArgumentException("minSalary phải <=
maxSalary");
        }

        Job job = new Job();
        job.id = UUID.randomUUID();
        job.title = title;
        job.minSalary = minSalary;
        job.maxSalary = maxSalary;
        job.status = Status.DRAFT;
        return job;
    }

    public void submitForApproval() {
        if (status != Status.DRAFT) {
            throw new IllegalStateException("Chỉ DRAFT mới submit
được");
        }
        this.status = Status.PENDING_APPROVAL;
    }

    public void approve() {
        if (status != Status.PENDING_APPROVAL) {
            throw new IllegalStateException("Chỉ duyệt được khi đang
PENDING_APPROVAL");
        }
        this.status = Status.APPROVED;
    }

    public void publish() {
        if (status != Status.APPROVED) {
            throw new IllegalStateException("Phải APPROVED mới publish
được");
```

```
        }
        this.status = Status.PUBLISHED;
    }
}
```

Cách dùng:

```
Job job = Job.draft("Java Developer", new BigDecimal("15000000"), new
BigDecimal("25000000"));
job.submitForApproval();   // DRAFT : PENDING_APPROVAL
job.approve();             // PENDING_APPROVAL : APPROVED
job.publish();             // APPROVED : PUBLISHED

job.publish();             // ← Throws IllegalStateException (đã
PUBLISHED)
Job.draft("Bug", new BigDecimal("-100"), new BigDecimal("200"));  // ←
Throws IllegalArgumentException
```

Ví dụ 2: Aggregate `Candidate` (candidate-service)

Quy tắc nghiệp vụ:

- Email phải hợp lệ + không rỗng
- CV phải upload trước khi parse
- Parsed CV chỉ set được khi đã có CV file

```
public class Candidate {

    private UUID id;
    private String fullName;
    private String email;
    private String cvFileUrl;
    private ParsedCvData parsedCvData;

    private Candidate() { }

    public static Candidate register(String fullName, String email) {
        if (email == null || email.isBlank() || !email.contains("@")) {
            throw new IllegalArgumentException("Email không hợp lệ");
        }
        Candidate c = new Candidate();
        c.id = UUID.randomUUID();
        c.fullName = fullName;
        c.email = email;
        return c;
    }

    public void attachCv(String cvUrl) {
        if (cvUrl == null || cvUrl.isBlank()) {
            throw new IllegalArgumentException("CV URL không hợp lệ");
        }
        this.cvFileUrl = cvUrl;
    }

    public void completeParsing(ParsedCvData data) {
        if (cvFileUrl == null) {
            throw new IllegalStateException("Phải upload CV trước khi
parse");
        }
        this.parsedCvData = data;
    }
}
```

Cách dùng:

```
Candidate c = Candidate.register("Nguyễn Văn A", "a@gmail.com");

// Lỗi: chưa upload CV
c.completeParsing(parsedData);   // ← Throws IllegalStateException

// Đúng flow
c.attachCv("https://minio/cv/123.pdf");
c.completeParsing(parsedData);   // ✅ OK

// Email sai
Candidate.register("B", "invalid-email");   // ← Throws
IllegalArgumentException
```

Ví dụ 3: Aggregate `Application` (application-service) - state machine phức tạp

Quy tắc nghiệp vụ (BRULE-12):

- Pipeline: `NEW : CV_SCREENING : INTERVIEW : OFFERED : HIRED`
- Không skip stage (vd: NEW : INTERVIEW = invalid)
- Không backward (trừ TRANSFERRED ở giai đoạn CV_SCREENING)
- REJECTED là terminal state

```
public class Application {

    private UUID id;
    private UUID candidateId;
    private UUID jobId;
    private PipelineStage currentStage;

    private Application() { }

    public static Application submit(UUID candidateId, UUID jobId) {
        if (candidateId == null || jobId == null) {
            throw new IllegalArgumentException("candidateId + jobId là
bắt buộc");
        }
        Application app = new Application();
        app.id = UUID.randomUUID();
        app.candidateId = candidateId;
        app.jobId = jobId;
        app.currentStage = PipelineStage.NEW;
        return app;
    }

    public void advanceStage(PipelineStage next) {
        if (!canAdvanceTo(next)) {
            throw new IllegalStateException(
                "Không thể chuyển từ " + currentStage + " sang " +
next);
        }
        this.currentStage = next;
    }

    private boolean canAdvanceTo(PipelineStage next) {
        return switch (currentStage) {
            case NEW -> next == PipelineStage.CV_SCREENING || next ==
PipelineStage.REJECTED;
            case CV_SCREENING -> next == PipelineStage.INTERVIEW || next
== PipelineStage.REJECTED;
            case INTERVIEW -> next == PipelineStage.OFFERED || next ==
PipelineStage.REJECTED;
            case OFFERED -> next == PipelineStage.HIRED;
            default -> false; // HIRED, REJECTED = terminal
        };
    }
}
```

Cách dùng:

```
Application app = Application.submit(candidateId, jobId);

// Đúng flow
app.advanceStage(PipelineStage.CV_SCREENING);   // ✅
app.advanceStage(PipelineStage.INTERVIEW);       // ✅
app.advanceStage(PipelineStage.OFFERED);         // ✅
app.advanceStage(PipelineStage.HIRED);           // ✅

// Skip stage - Throws IllegalStateException
Application app2 = Application.submit(c, j);
app2.advanceStage(PipelineStage.INTERVIEW);   // ❌ NEW không thể nhảy
thẳng INTERVIEW

// Backward - Throws IllegalStateException
app.advanceStage(PipelineStage.NEW);   // ❌ HIRED là terminal
```

: **Application Aggregate tự bảo vệ chuỗi pipeline** - không ai bypass được state machine.

---

5. Khi nào dùng Aggregate Root?

Hỏi 3 câu trước khi quyết định:

Câu hỏi	Nếu YES : là Aggregate Root
Object này có **ID/identity** riêng không?	✅ (Job có UUID, OrderLine không cần ID riêng)
Bên ngoài có **tìm kiếm/query** trực tiếp object này không?	✅ (`jobRepo.findById(jobId)`)
Object này có **quy tắc nghiệp vụ** cần bảo vệ không?	✅ (BRULE-11, BRULE-14)

: Job, Candidate, Application, NotificationLog, CvParseResult - tất cả đều có 3 đặc điểm trên : **đều là Aggregate Root**.

---

6. Quy tắc chung khi viết Aggregate ở stage1

Quy tắc	Lý do
Pure POJO - không annotation JPA/Spring	L3 chỉ tập trung domain, persistence sẽ thêm ở L4
Constructor private + `static factory method`	Kiểm soát cách tạo object - luôn ở state hợp lệ
Không public setter	Tránh ai cũng sửa state lung tung
Method có tên ý nghĩa nghiệp vụ (`approve()` , `publish()` , `cancel()`)	Code đọc như đọc requirement
Throw exception khi vi phạm quy tắc	Fail-fast - bug lộ ngay

7. Tóm tắt 1 dòng

> **Aggregate Root = object có ID + chứa quy tắc nghiệp vụ tự bảo vệ + thay đổi state qua method tên ý nghĩa (KHÔNG setter). Mỗi service trong TalentHub có ít nhất 1 Aggregate Root.**

8. Roadmap tiến hoá ở các stage sau

Stage	Aggregate sẽ phát triển ra sao
stage 1 (L3)	POJO thuần - chỉ field + factory + behavior method. Chưa có `@Entity` , chưa lưu DB.
stage 2 (L5)	Workshop - vẽ Context Map xác định Aggregate boundary
stage 3 (L6)	**Gắn `@Entity` + `@Id` (JPA)** để map xuống Postgres - nhưng VẪN giữ cấu trúc Rich Model (private constructor, no public setter, business method trên Root). Đây là lúc "POJO trở thành JPA Entity".
stage 6 (L9)	Nhiều Aggregate giao tiếp qua OpenFeign (inter-service call). Entity vẫn giữ Rich Model.

Stage	Aggregate sẽ phát triển ra sao
L17 nâng cao	Thêm `registerEvent()` cho Outbox pattern + Saga compensation

> **Lưu ý quan trọng:**
>
> - Ở stage1, **KHÔNG cần JPA, KHÔNG cần lưu DB** - chỉ tập trung dịch sticky note Bounded Context thành Java class với quy tắc nghiệp vụ rõ ràng.
> - Từ stage3 (L6), Aggregate được gắn `@Entity` nhưng **bản chất vẫn là Rich Model** - class vẫn chứa business logic, validation, state machine. **Không biến thành Anemic Model** chỉ vì thêm JPA annotation.
> - Cách tiếp cận này gọi là **Pragmatic DDD** - chấp nhận nhúng annotation JPA vào Domain Model để giảm boilerplate, thay vì tách riêng Domain Model và Persistence Entity (Pure DDD/Hexagonal). Đây là cách được ưa chuộng trong thực tế dự án Spring Boot.

Version: v1.0